

**LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III**



Acara 9

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026**

Bab 1. Pendahuluan

1.1 latar Belakang

Dalam pengembangan aplikasi berbasis PHP, pengelolaan kode yang baik diperlukan agar aplikasi mudah dipahami, diuji, dan dikembangkan. Salah satu permasalahan yang dapat muncul adalah ketika Controller membuat koneksi database secara langsung atau memiliki terlalu banyak tanggung jawab. Kondisi tersebut menyebabkan kode menjadi lebih sulit diuji dan sulit dikembangkan apabila terjadi perubahan pada implementasi database.

Pada praktikum Acara 9 dilakukan refactoring pada aplikasi Manajemen Data Mahasiswa dengan menerapkan konsep **Object Composition**, **Constructor Dependency Injection**, serta **Getter dan Setter**.

Object Composition merupakan konsep pemrograman berorientasi objek yang mengutamakan hubungan "memiliki" (*has-a*) antarobjek dibandingkan penggunaan pewarisan (*inheritance*) yang terlalu dalam. Pada aplikasi ini, Database memiliki objek PDO, MahasiswaRepository memiliki koneksi database, dan MahasiswaController memiliki objek MahasiswaRepository.

Selanjutnya diterapkan **Dependency Injection**, yaitu memberikan objek yang dibutuhkan oleh suatu class dari luar class tersebut. Pada praktikum ini, MahasiswaRepository menerima objek Database melalui constructor, sedangkan MahasiswaController menerima MahasiswaRepository melalui constructor. Dengan cara tersebut, Controller tidak membuat koneksi database secara langsung.

Konsep **Getter dan Setter** juga diterapkan pada class Mahasiswa. Seluruh atribut mahasiswa dibuat menjadi private, kemudian disediakan method getter untuk membaca data dan setter untuk mengubah data. Setter juga digunakan untuk menambahkan validasi sederhana, seperti memastikan nama mahasiswa tidak kosong dan NIM hanya terdiri dari angka. Dengan penerapan ketiga konsep tersebut, aplikasi menjadi lebih terstruktur karena setiap class memiliki tanggung jawab yang lebih jelas. Selain itu, kode menjadi lebih fleksibel, mudah diuji, dan lebih mudah dikembangkan.

1.2 Tujuan Praktikum

1. Memahami konsep **Object Composition** dalam pengembangan aplikasi berbasis PHP.
2. Memahami dan menerapkan **Dependency Injection (DI)** menggunakan constructor injection.
3. Memisahkan tanggung jawab antara **Controller, Repository, Model, dan Database** agar kode lebih terstruktur.

4. Menerapkan **Getter dan Setter** untuk mengatur akses serta validasi terhadap atribut pada Model Mahasiswa.
5. Menerapkan **validasi data** seperti NIM, nama, email, program studi, dan angkatan.
6. Memastikan fitur **CRUD (Create, Read, Update, Delete)** mahasiswa tetap berjalan setelah dilakukan refactoring.
7. Membuat kode yang lebih **fleksibel, mudah dipelihara, dan mudah diuji**.

Bab 2. Dasar Teori

2.1 Object Composition

Object Composition adalah teknik menyusun sebuah class dengan cara memiliki atau menggunakan objek dari class lain. Konsep ini dikenal sebagai hubungan *has-a*.

Pada aplikasi praktikum, penerapannya dapat dilihat pada class Database yang memiliki objek PDO:

```
private PDO $pdo;
```

Selain itu, MahasiswaRepository memiliki koneksi database dan MahasiswaController memiliki objek MahasiswaRepository.

2.2 Depedency Injection

Dependency Injection adalah teknik memberikan dependency atau objek yang dibutuhkan suatu class dari luar class tersebut. Pada praktikum digunakan **Constructor Dependency Injection**.

Contohnya pada MahasiswaRepository:

```
public function __construct(Database $database)
{
    $this->db = $database->getConnection();
}
```

Object Database diberikan melalui constructor sehingga Repository tidak perlu membuat object Database sendiri.

2.3 Getter dan Setter

Getter digunakan untuk mengambil nilai atribut private, sedangkan Setter digunakan untuk mengubah nilai atribut private. Setter juga dapat digunakan untuk melakukan validasi sebelum data disimpan. Dengan cara tersebut, atribut tidak dapat diubah secara langsung dan setiap perubahan data dapat dikontrol melalui setter.

Bab 3. Hasil Praktikum

3.1 Pembuatan Class database

Tahap pertama adalah melakukan refactoring pada class Database. Class ini bertugas mengelola koneksi database menggunakan PDO.

Pada class Database, object PDO disimpan sebagai atribut private:

```
private PDO $pdo;
```

Koneksi database dibuat pada constructor dan kemudian dapat diakses melalui method getConnection().

```
public function getConnection(): PDO
```

```
{
```

```
    return $this->pdo;
```

```
}
```

Dengan demikian, class lain tidak perlu membuat koneksi PDO sendiri.

si-akademik > app > Core > Database.php > ...

```
3 namespace App\Core;
  11 references | 0 implementations
8 class Database
9 {
  3 references
10     private static ?Database $instance = null;
11
  2 references
12     private PDO $pdo;
13
  1 reference
14     private function __construct()
15     {
16         $config = require __DIR__ . '/../../config/database.php';
17
18         $dsn = "mysql:host={$config['host']};dbname={$config['dbname']}";
19
20         try {
21             $this->pdo = new PDO(
22                 $dsn,
23                 $config['username'],
24                 $config['password'],
25                 [
26                     PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
27                     PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
28                     PDO::ATTR_EMULATE_PREPARES => false,
29                 ]
            )
        }
```

3.2 Pembuatan Mahasiswa Repository

```
MahasiswaRepository.php X
si-akademik > app > Repositories > MahasiswaRepository.php > ...

3 namespace App\Repositories;
9 class MahasiswaRepository

    8 references
11     private PDO $db;
12
    1 reference | 0 overrides
13     public function __construct(Database $database)
14     {
15         $this->db = $database->getConnection();
16     }
17
    1 reference | 0 overrides
18     public function all(
19         ?string $keyword = null,
20         int $limit = 5,
21         int $offset = 0
22     ): array {
23         $sql = "
24             SELECT
25                 m.id,
26                 m.nim,
27                 m.nama,
28                 m.email,
29                 m.prodi_id,
30                 m.angkatan,
31                 m.status,
32                 m.updated_at,
33
```

3.3 Penerepan Getter dan Setter pada Class Mahasiswa

```
1  <?php
2
3  namespace App\Models;
4
5  use InvalidArgumentException;
6
7  6 references | 0 implementations
8  class Mahasiswa
9  {
10     2 references
11     private ?int $id = null;
12     2 references
13     private string $nim = '';
14     2 references
15     private string $nama = '';
16     2 references
17     private string $email = '';
18     2 references
19     private int $prodiId = 0;
20     2 references
21     private int $angkatan = 0;
22     2 references
23     private string $status = 'aktif';
24 }
```

```
71
72  2 references | 0 overrides
73  public function setNim(string $nim): void
74  {
75      $nim = trim($nim);
76
77      if ($nim === '') {
78          throw new InvalidArgumentException(
79              'NIM tidak boleh kosong.'
80          );
81      }
82
83      if (!ctype_digit($nim)) {
84          throw new InvalidArgumentException(
85              'NIM harus berupa angka.'
86          );
87      }
88
89      $this->nim = $nim;
90 }
```

```

2 references | 0 overrides
96  ✓ public function setName(string $nama): void
97  {
98      $nama = trim($nama);
99
100  ✓   if ($nama === '') {
101      |       throw new InvalidArgumentException(
102      |           'Nama mahasiswa tidak boleh kosong.'
103      |       );
104      |   }
105
106      $this->nama = $nama;
107  }
108

```

3.4 Validasi NIM sesuai Tahun Angkatan

```

334  ✓ private function validateInput(): ?Mahasiswa
406
407      $prefixAngkatan = substr(
408          (string) $mahasiswa->getAngkatan(),
409          -2
410      );
411
412      if (
413          substr(
414              $mahasiswa->getNim(),
415              0,
416              2
417          ) !== $prefixAngkatan
418      ) {
419          return null;
420      }
421
422
423      return $mahasiswa;
424
425  } catch (
426      InvalidArgumentException $e
427  ) {
428
429      return null;

```

3.5 Refactoring Mahasiswa Controller

```
18     private MahasiswaRepository $repo;
19
20
21     /*
22     * =====
23     * CONSTRUCTOR DEPENDENCY INJECTION
24     * =====
25     *
26     * Repository diberikan dari luar.
27     */
28     1 reference | 0 overrides
29     public function __construct(
30         MahasiswaRepository $repo
31     ) {
32         $this->repo = $repo;
33     }
```

3.6 Dependency Injection pada public/index.php

```
122     /*
123     * 1. Buat Database.
124     */
125     $database =
126         Database::getInstance();
127
128
129     /*
130     * 2. Masukkan Database ke Repository.
131     *
132     * Constructor Dependency Injection.
133     */
134     $mahasiswaRepository =
135         new MahasiswaRepository(
136             $database
137         );
138
139
140     /*
141     * 3. Masukkan Repository ke Controller.
142     *
143     * Constructor Dependency Injection.
144     */
145     $controllerFactories = [
```

3.7 Pengujian Menampilkan Data Mahasiswa

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Data Mahasiswa

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	250004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	TK - Teknik Komputer	2025	Aktif	DetailEditHapus
2	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	DetailEditHapus
3	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	DetailEditHapus
4	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	DetailEditHapus

Menampilkan 4 dari 4 data mahasiswa.

← Kembali ke Dashboard

3.8 Pengujian Tambah Data

Tambah Data Mahasiswa

NIM

250004

Nama Mahasiswa

Nila Agustin Prima Aurelia

Email

nilaaurelia97@gmail.com

Program Studi

TI - Teknik Informatika

Angkatan

2025

Status Mahasiswa

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Data Mahasiswa

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	250004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	TI - Teknik Informatika	2025	Aktif	DetailEditHapus
2	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	DetailEditHapus
3	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	DetailEditHapus
4	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	DetailEditHapus

Menampilkan 4 dari 4 data mahasiswa.

← Kembali ke Dashboard

3.9 Pengujian Validasi

- Nama Kosong

The first screenshot shows the 'Tambah Data Mahasiswa' form with the following data:

- NIM: 250005
- Nama Mahasiswa: (empty)
- Email: ingka27@gmail.com
- Program Studi: TI - Teknik Informatika
- Angkatan: 2025
- Status Mahasiswa: Aktif

The second screenshot shows the same form, but the 'Nama Mahasiswa' field is highlighted with a red border, and an error message is displayed: "Please fill out this field."

- NIM Bukan Angka

The first screenshot shows the 'Tambah Data Mahasiswa' form with the following data:

- NIM: E250005
- Nama Mahasiswa: Ingka

The second screenshot shows the same form, but a red error message is displayed at the bottom: "Data tidak valid. Pastikan nama tidak kosong, NIM berupa angka, dan 2 angka pertama NIM sesuai tahun angkatan."

- NIM dan Angkatan

Tambah Data Mahasiswa

NIM

250005

Nama Mahasiswa

Ingka

Email

ingka27@gmail.com

Program Studi

TI - Teknik Informatika

Angkatan

2026

Status Mahasiswa

Aktif

Kembali

Simpan

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Data tidak valid. Pastikan nama tidak kosong, NIM berupa angka, dan 2 angka pertama NIM sesuai tahun angkatan.

3.10 Pengujian Edit Data

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Edit Data Mahasiswa

NIM

260004

Data mahasiswa berhasil diubah.

Data Mahasiswa

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	260004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	TI - Teknik Informatika	2026	Aktif	DetailEditHapus

3.11 Pengujian Hapus Data

SI Akademik

localhost says
Yakin ingin menghapus data mahasiswa ini?

Yakin ingin menghapus data mahasiswa ini?

OKCancel

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	260004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	TI - Teknik Informatika	2026	Aktif	Detail Edit Hapus
2	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	Detail Edit Hapus
3	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus
4	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Data mahasiswa berhasil dihapus.

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	Detail Edit Hapus
2	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus
3	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

3.12 Pengujian Pencarian

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

+ Tambah Mahasiswa

budi

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

Menampilkan 1 dari 1 data mahasiswa.

Kembali ke Dashboard

© 2026 SI Akademik - Politeknik Negeri Jember

Bab 4. Kesimpulan

Berdasarkan praktikum Acara 9 yang telah dilakukan, dapat disimpulkan bahwa penerapan Object Composition, Constructor Dependency Injection, serta Getter dan Setter dapat membuat struktur aplikasi menjadi lebih terorganisir.

Object Composition diterapkan dengan membuat objek saling memiliki atau menggunakan objek lain, yaitu Database memiliki objek PDO, MahasiswaRepository menggunakan koneksi database, dan MahasiswaController menggunakan MahasiswaRepository.

Constructor Dependency Injection diterapkan dengan memberikan objek Database kepada MahasiswaRepository melalui constructor dan memberikan MahasiswaRepository kepada MahasiswaController melalui constructor. Dengan pendekatan tersebut, Controller tidak lagi membuat koneksi database secara langsung sehingga kode menjadi lebih fleksibel dan mudah dikembangkan.

Getter dan Setter diterapkan dengan membuat seluruh atribut pada class Mahasiswa menjadi private. Akses dan perubahan data dilakukan melalui method getter dan setter. Setter juga digunakan untuk melakukan validasi, seperti memastikan nama mahasiswa tidak kosong dan NIM hanya berupa angka.

Setelah proses refactoring, fitur utama aplikasi seperti menampilkan, menambah, mengubah, menghapus, dan mencari data mahasiswa tetap dapat berjalan. Dengan demikian, tujuan praktikum untuk memahami penerapan Object Composition, Constructor Dependency Injection, serta Getter dan Setter telah tercapai.